

FE Code Convention

Code Convention

Next.js Frontend Application - ASUM FE

- [Code Convention](#)
 - [1. Purpose & Scope](#)
 - [2. Module Architecture Standard](#)
 - [2.1 Standard Module Structure](#)
 - [3. File Responsibility & Examples](#)
 - [3.1 ModuleName.component.tsx](#)
 - [3.2 ModuleName.container.tsx](#)
 - [3.3 ModuleName.wrapper.tsx](#)
 - [3.4 ModuleName.type.ts](#)
 - [3.5 ModuleName.config.ts](#)
 - [3.6 ModuleName.style.ts/ ModuleName.css](#)
 - [3.7 ModuleName.hook.ts](#)
 - [3.8 ModuleName.component.test.ts](#)
 - [3.9 ModuleName.component.snap.ts](#)
 - [3.10 index.ts](#)
 - [3.11 ModuleName.utils.ts](#)
 - [3.12 ModuleName.helpers.ts || ModuleName.helpers.tsx](#)
 - [4. Enterprise Coding Rules](#)
 - [4.1 Formatting & Style](#)
 - [4.2 Hooks Policy](#)
 - [4.3 JSDoc Typing Policy](#)
 - [4.4 Event Handler Policy](#)
 - [5. Testing Policy](#)
 - [6. Shared Code Rules](#)
 - [6.1 Common Types](#)
 - [6.2 Shared Utilities](#)
 - [7. API Integration Standard](#)
 - [8. Naming Variables & Functions](#)
 - [8.1 🧠 General Principles](#)
 - [8.2. 📄 Variable Naming](#)
 - [✅ Use camelCase](#)
 - [❌ Avoid](#)
 - [8.3. 🧩 Boolean Variables → use prefixes:](#)
 - [📦 Arrays → use plural form](#)
 - [🔢 Constants → use UPPER_SNAKE_CASE](#)
 - [8.3. ⚙️ Function Naming](#)
 - [✅ Use camelCase](#)
 - [🎯 Use prefixes based on purpose](#)
 - [🔍 Getter / Fetching](#)
 - [➕ Create / Actions](#)
 - [🖋️ Update](#)
 - [❌ Delete](#)
 - [✔️ Boolean Return → use is, has, can](#)
 - [8.4. 📄 Object & Class Naming](#)
 - [Class → PascalCase](#)
 - [Object Variables → still camelCase](#)
 - [8.5. 🚫 Context-Based Naming \(Best Practice\)](#)

- [🔗 API / Backend](#)
 - [🔧 Testing \(Jest / Unit Test\)](#)
 - [Use format:](#)
 - [8.6. 🚫 What to Avoid](#)
 - [8.7. 💡 Additional Tips](#)
 - [9. Git](#)
 - [Commit and prefix branch Type Guide:](#)
 - [10. Enforcement](#)
 - [11. Conclusion](#)
-

1. Purpose & Scope

This document defines the **official enterprise coding standards** for Next.js frontend development.

It is intended to:

- Enforce **consistency** across teams
- Improve **maintainability and scalability**
- Reduce **technical debt**
- Ensure **testability and code quality**

This convention **MUST** be followed by all contributors.

2. Module Architecture Standard

2.1 Standard Module Structure

Every module **MUST** follow this directory structure:

```
1  ModuleName/  
2  |  ModuleName.component.tsx  
3  |  ModuleName.component.snap.ts  
4  |  ModuleName.component.test.ts  
5  |  ModuleName.container.tsx  
6  |  ModuleName.container.snap.tsx  
7  |  ModuleName.container.test.ts  
8  |  ModuleName.type.ts  
9  |  ModuleName.config.ts  
10 |  ModuleName.style.ts // optional  
11 |  ModuleName.wrapper.tsx  
12 |  ModuleName.hook.ts  
13 |  ModuleName.utils.ts // optional  
14 |  ModuleName.helpers.ts // // optional  
15 |  index.ts  
16
```

3. File Responsibility & Examples

3.1 ModuleName.component.tsx

Responsibility

- Pure UI component
- No API calls
- No Redux / side effects
- Receives data via props only
- Accept Event Handler

Example

```

1  import * as React from 'react';
2
3  import { UserCardProps } from './ModuleName.type';
4  import style from './ModuleName.style';
5
6  /**
7   * Render Header Card
8   * @param {string} title - header title
9   * @returns {React.ReactElement} - Header
10  */
11  const _renderHeader = (title: string): React.ReactElement => (
12    <h3>{title}</h3>
13  );
14
15  /**
16   * Render Content Card
17   * @param {string} email - card content
18   * @returns {React.ReactElement} - Header
19  */
20  const _renderContent = (email: string): React.ReactElement => (
21    <p>{email}</p>
22  );
23
24  /**
25   * Render UserCard Component
26   * @param {UserCardProps} props - user card component props
27   * @returns {React.ReactElement} - UserCardComponent
28  */
29  export const UserCardComponent = (props: UserCardProps):
    React.ReactElement => {
30    const { title, email } = props;
31
32    return (
33      <Card>
34        {_renderHeader(title)}
35        {_renderContent(email)}
36      </Card>
37    );
38  };
39

```

3.2 ModuleName.container.tsx

Responsibility

- Business logic
- Redux / RTK Query (If you consume a lot of hooks, it's better to move the hooks to the .hook.ts file)
- API interaction
- Prop mapping

Example

```
1 import * as React from 'react';
2 import { connect } from 'react-redux';
3
4 import { useGetUserQuery } from '@store/api/user.api';
5
6 import { UserCardComponent } from './ModuleName.component';
7 import type { Props } from './ModuleName.type';
8
9 /**
10  * Render UserCard Container
11  * @param {Props} props - props
12  * @returns {React.ReactElement} - UserCard Container
13  */
14 const UserCardContainer = (props: Props) => {
15   const { data } = useGetUserQuery();
16
17   return (
18     <UserCardComponent
19       name={data?.name ?? ''}
20       email={data?.email ?? ''}
21     />
22   );
23 };
24
25 const mapSatateToProps = (state) => ({});
26 const mapDispatchToProps = (dispatch) => ({});
27
28 export default connect(mapSatateToProps, mapDispatchToProps)
29   (UserCardContainer);
```

3.3 ModuleName.wrapper.tsx

Responsibility

- Server ↔ Client Component bridge
- Place of **SSR** implementation
- Used for Next.js App Router

Example

```
1 'use server'
2
3 import ModuleNameContainer from './ModuleName.container';
4
5 const ModuleNameContainerWrapper = async () => {
6   return <ModuleNameContainer />;
7 };
8
9 export default ModuleNameContainerWrapper;
```

3.4 ModuleName.type.ts

Responsibility

- Centralized typing
- You can only define type here

- No logic
- No JSX

Example Type

```
1 export type UserCardProps = {  
2   name: string;  
3   email: string;  
4 };  
5
```

❌ Avoid using Interface and Ambiguous shape (*, any, unknown, Record<string, unknown>, etc).

```
1 # forbidden data type  
2  
3 ## Record<string, unknown> - object with a free shape  
4 type User = Record<string, unknown>;  
5  
6 const userData: User = {  
7   name: "Januar",  
8   age: 25,  
9   active: true,  
10 };  
11  
12 ## Index signature - the key is known as a string, the value has a  
13   certain type  
14 type UserData = {  
15   [key: string]: string | number | boolean | null | undefined;  
16 };  
17  
18 const user: UserData = {  
19   name: "Januar",  
20   age: 25,  
21   active: true,  
22 };  
23  
24 ## unknown - the shape/value is really unknown  
25 type Data = unknown[];  
26 let data: unknown;  
27  
28 data = "hello";  
29 data = 123;  
30 data = { name: "Januar" };  
31  
32 ## any - Anything goes  
33 let data: any;  
34  
35 data = "hello";  
36 data = 123;  
37 data = { name: "Januar" };  
38  
39 ## object - Object generik  
40 let data: object;  
41  
42 ## Type Assertion  
43 type User = {  
44   name: string,  
45   age: number  
46 }  
47  
48 const value = {  
49   address: 'Jakarta',  
50   street: 'Jl Juanda'  
51 } as User;
```

3.5 ModuleName.config.ts

Responsibility

- Static configuration
- Constants
- Feature flags
- Keep .config.ts purely static: Never put logic that returns JSX (React.createElement or HTML tags) inside .config.ts files. namemodule .config.ts file must contain purely static data or constants. If a dynamic builder function requires React imports to render UI elements, it belongs in .helpers.tsx.

Example

```
1  ## only constant
2  export const USER_CARD_CONFIG = {
3    MAX_NAME_LENGTH: 50,
4  };
5
```

❌ Avoid any logic

3.6 ModuleName.style.ts / ModuleName.css

Responsibility

- Styling only
- No logic

Example

```
1  // for inline style
2  import { CSSProperties } from 'react';
3
4  const styles: Record<string, CSSProperties> = {
5    container: {
6      flex: 1,
7      padding: 16,
8      backgroundColor: '#f5f5f5',
9    },
10   header: {
11     fontSize: 24,
12     fontWeight: 'bold',
13     marginBottom: 16,
14   },
15   button: {
16     backgroundColor: '#007bff',
17     padding: 12,
18     borderRadius: 4,
19     alignItems: 'center',
20   },
21   buttonText: {
22     color: 'ffffff',
23     fontSize: 16,
24     fontWeight: '600',
25   },
26 };
27
```

```
28 export default styles;
29
```

3.7 ModuleName.hook.ts

Responsibility

- Encapsulate `useState` and `useEffect`
- If your component has a lot of hooks move them here
- Reusable logic
- If the hook is reusable then create the hook in the `@src/hooks` folder

Example

```
1 import { useEffect, useState } from 'react';
2
3 export const useUserState = () => {
4   const [isActive, setIsActive] = useState(false);
5
6   return { isActive };
7 };
8
9 export const useUserEffect = () => {
10  useEffect(() => {
11    setIsActive(true);
12  }, []);
13 }
14
```

3.8 ModuleName.component.test.ts

Responsibility

- Unit testing Functional by scenario

Example

```
1 import {
2   _calculate,
3   from './ModuleName.component';
4
5 jest.mock('module', () => 'module');
6
7 describe('Module Name Test', () => {
8   it('Should calculate a + b', () => {
9     const mockA = 1;
10    const mockB = 2;
11
12    const result = _calculate(mockA, mockB)
13
14    expect(result).toBe(3);
15  });
16 });
17
```

3.9 ModuleName.component.snap.ts

Responsibility

- Snapshot testing by scenario

Example

```
1 import test from '@libs/unit-test';
2
3 import UserCardComponent from './ModuleName.component';
4
5 const configs = [
6   {
7     props: {},
8     desc: 'Should Render UserCardComponent with default props',
9     useHook: true
10  },
11  ...
12 ]
13
14 it('ModuleName matches snapshot', () => {
15   test.assertSnapshots(UserCardComponent, configs);
16 });
17
```

3.10 index.ts

Responsibility

- Public module exports
- Clean import interface

Example

```
1 export { default } from './ModuleName.wrapper';
2
```

3.11 ModuleName.utils.ts

Responsibility

- Private utils exports (NON GENERIC)
- Just for the needs of the related component/module
- don't put generic utils / shared global utils here
- distinguish between local utils and shared (global) utils

3.12 ModuleName.helpers.ts || ModuleName.helpers.tsx

- **Private helpers exports (NON GENERIC):** Just for the needs of the related component/module.
- **Don't put generic helpers / shared global helpers here:** Distinguish between local helpers and shared (global) helpers.
- **When to use .ts vs .tsx :**

- Use `.helpers.ts` for standard logic helper functions (e.g., data formatting, calculations, mappers) that are pure JavaScript/TypeScript.
 - Use `.helpers.tsx` if the helper function needs to return a JSX Element or React Node (e.g., builder functions for custom modal messages, dynamic alert texts, or custom renderers for table columns).
-

4. Enterprise Coding Rules

4.1 Formatting & Style

- One blank line at the end of every file
 - Semicolons are mandatory
 - Blank line before:
 - `return`
 - `if`
 - `switch`
 - `while`
-

4.2 Hooks Policy

-  `useState` inside component → NOT allowed
 -  `useEffect` inside component → NOT allowed
 -  Must be moved to hook files
-

4.3 JSDoc Typing Policy

- JSDoc is mandatory for public functions
- Explicit return types are mandatory
- `any` and `*` are strictly prohibited
- Add function description
- JsDoc Format Name, type, and description, with a hyphen before the description
- place optional params at the end of function params

Example JS Doc

```
1 /**
2  * Render Function Calculate
3  * @param {number} a - a value
4  * @param {number} b - b value
5  * @param {number} [c] - c is optional params
6  * @returns {number} - Result calculate
7  */
8 const _calculate = (a: number, b: number, c?: number): number => a + b;
9
```

JS Doc ❌ Incorrect

```
1  /**
2   * Render Function Calculate
3   * @param {number} a - a value
4   * @param {number} b - b value
5   * @param {number | undefined} c - c is optional params ❌ Incorrect
6   * @returns {number} - Result calculate
7   */
8  const _calculate = (a: number, b: number, c?: number): number => a +
  b;
9
10 /**
11  * Render Function Calculate
12  * @param {number} a - a value
13  * @param {number | undefined} c - c is optional params ❌ Incorrect
14  * @param {number} b - b value
15  * @returns {number} - Result calculate
16  */
17  const _calculate = (a: number, c?: number, b: number,): number => a +
  b;
18
19 /**
20  * Render Function Calculate
21  * @param {User} user - a value
22  * @param {User} user.userName - ❌ Incorrect
23  * @param {User} user.email - ❌ Incorrect
24  * @returns {string} - User Name
25  */
26  const _getUserName = (user: User): string => user.Username;
27
28 /**
29  * Render Function Calculate
30  * @param {{userName: string, email: string}} user - ❌ Incorrect
31  * @returns {string} - User Name
32  */
33  const _getUserName = (user: User): string => user.Username;
34
35 /**
36  * Render Function Calculate
37  * @param {object | any | * | unknown} user - ❌ Incorrect
38  * @returns {string} - User Name
39  */
40  const _getUserName = (user: User): string => user.Username;
```

4.4 Event Handler Policy

❌ Incorrect

```
1  <Button onClick={() => handleClick()}>Submit</Button>
```

✅ Correct

```
1  <Button onClick={handleClick}>Submit</Button>
```

5. Testing Policy

- Jest is mandatory
- Snapshot + Unit tests are mandatory
- Minimum coverage: **≥ 70%**

- PR will be rejected if coverage is below threshold

6. Shared Code Rules

6.1 Common Types

```
1 | type/common.type.ts
```

6.2 Shared Utilities

```
1 | libs/utils
```

7. API Integration Standard

- API calls **MUST** use:
 - RTK Query
 - Axios-based

```
1 | import apiClient from '@libs/api/ApiClient/ApiClient';
2 | import {
3 |   useGetByUrlQuery,
4 |   useLazyGetByUrlQuery,
5 |   useMutateByUrlMutation
6 | } from '@libs/ApiRtkServices/genericEndpoints.service';
```

- Direct `fetch` usage is prohibited

8. Naming Variables & Functions

8.1 🧠 General Principles

- Use **English**
- Names must be **descriptive and meaningful**
- Avoid unclear abbreviations (`usr` , `dt` , `val`)
- Keep naming **consistent** across the project

⚠️ Avoid using `var` if you don't really need hoisting variable.

8.2. 📄 Variable Naming

✅ Use `camelCase`

```
1 | const userName = "Camel Case";
2 | const totalPrice = 10000;
3 | const isLoggedIn = true;
```

❌ Avoid

```
1 | const username = "Subhan Singandaru"; // unclear combined words
```

```
2 const usr = "Subhan"; // too short
3 const x = 10000; // not meaningful
```

🌱 Boolean Variables → use prefixes:

- is
- has
- can
- should

```
1 const isActive = true;
2 const hasPermission = false;
3 const canEdit = true;
4 const shouldReload = false;
```

📦 Arrays → use plural form

```
1 const users = [];
2 const products = [];
3 const orders = [];
```

🔢 Constants → use UPPER_SNAKE_CASE

```
1 const MAX_RETRY = 3;
2 const API_BASE_URL = "https://api.example.com";
```

8.3. ⚙️ Function Naming

✅ Use camelCase

⚠️ Better using Arrow Function (Recommended)

Avoid using function declaration if you don't really need hoisting function.

```
1 function getUserData() {}
2 function calculateTotalPrice() {}
3 function fetchOrders() {}
4 const getCompanyList = () => {}
5 const _renderCompanyList = () => {} // add the prefix _ (underscore) if
  it is a private function
6 const useListEffect = () => {} // add the `use` prefix if the function
  is a hook.
```

🎯 Use prefixes based on purpose

🔍 Getter / Fetching

```
1 function getUser() {}
2 function fetchProducts() {}
3 function loadProfile() {}
```

+ Create / Actions

```
1 function createUser() {}
```

```
2 function submitForm() {}
3 function sendEmail() {}
```

 Update

```
1 function updateProfile() {}
2 function editUser() {}
```

 Delete

```
1 function deleteUser() {}
2 function removeItem() {}
```

✓ Boolean Return → use `is`, `has`, `can`

```
1 function isValidEmail() {}
2 function hasAccess() {}
3 function canEditProfile() {}
```

8.4. Object & Class Naming

Class → `PascalCase`

```
1 class UserService {}
2 class PaymentController {}
3 class AuthRepository {}
```

Object Variables → still `camelCase`

```
1 const userProfile = {
2   firstName: "Brilliant",
3   lastName: "Andrie"
4 };
```

8.5. Context-Based Naming (Best Practice)

 API / Backend

```
1 const requestPayload = {};
2 const responseData = {};
3 const errorMessage = "";
```

 Testing (Jest / Unit Test)

Use format:

 `should + expected behavior`

```
1 it("should return user data when API succeeds", () => {});
2 it("should throw an error when token is invalid", () => {});
```

8.6. What to Avoid

 Inconsistent naming

```
1 get_user_data()
2 getUserData()
```

```
3 getUserData()
```

✗ Non-descriptive names

```
1 | const data = {};  
2 | const temp = {};
```

✗ Overly verbose names

```
1 | const userDataFromAPIResponseObject = {}; // too long
```

8.7. 💡 Additional Tips

- Use **singular vs plural correctly**
- Use **domain-specific language** (e.g., `premiumAmount`, `deductibleValue`)
- Avoid generic names like `data`, `item` unless context is very clear

9. Git

Commit and prefix branch Type Guide:

- `fix` : Bug fix.
- `feat` : Addition of a new feature.
- `docs` : Documentation changes only.
- `style` : Changes that do not affect the meaning of the code (spaces, formatting, semicolons).
- `refactor` : Code changes that are neither bug fixes nor new features.
- `perf` : Code changes to improve performance.
- `test` : Adding or fixing tests (test suite).
- `chore` : Updates to build tasks, package manager configs, etc.

Example:

Branch ⇒ `feat/IIAU-123-implement-JWT-refresh-token`

Commit message ⇒

- `feat(product-config): [IIAU-123] - implement jwt refresh token`
- `feat(master-product-config): [IIAU-123] - implement jwt refresh token`
- `feat(case-management): [IIAU-123] - implement jwt refresh token`
- `feat(components): [IIAU-123] - implement jwt refresh token`
- `feat(new-business): [IIAU-123] - implement jwt refresh token`

Description PR ⇒ Example

Link Ticket: [IIAU-672](#)

Summary:

Add document template preview with View Detail button on Product Detail page and include Include column on Underwriting Data detail coverage table

Purpose / Background:

Product Config detail page sebelumnya cuma nampilin informasi dokumen sebagai plain text tanpa bisa preview template document. PR ini nambahin "View Detail" button buat tiap tipe dokumen yang bisa fetch & render template preview via DocumentComposerPdfPreview. Plus nambahin kolom "Include" yang missing di tabel coverage Underwriting Data detail view.

Key Changes:

1. Document Detail — View Detail with Preview —
DocumentDetail.component.tsx: Extracted ViewDetailButton component with loading indicator + DocumentComposerPdfPreview modal; document items
sekarang render dynamic key labels dari config instead of single hardcoded values; items generated via getDocDetailConfig(config) instead of 7 hardcoded entries
2. Document Detail Config — DocumentDetail.config.ts (new): Created parseConfigAsDetailConfig & getDocDetailConfig buat reuse form document configuration (title, url, previewPayload, keysLabel, keyValue) di detail view
3. Document Detail Container — DocumentDetail.container.tsx: Added previewHandler for async document fetch on "View Detail" click, ViewDetailButtonContainer to manage preview state, & _renderViewDetailButton render prop pattern
4. Document Detail Types — DocumentDetail.type.ts: Added generics across props (DocumentDetailComponentProps, DetailConfigType, ViewDetailButtonProps, ViewDetailRenderProps), removed

unused idx prop

5. Product Config Detail — Cleanup — ProductConfigDetail.component.tsx:
Removed unused idx prop dari DocumentDetailContainer
6. Document Form — Reusable Config — Document.config.tsx: Renamed config functions jadi exported `_policyConfig` etc.; data parameter jadi optional buat reuse di detail context; added `blankDocumentData` constant & `DocumentConfigsType` return type
7. Document Form — Generics & Cleanup — Document.component.tsx, Document.container.tsx, Document.type.ts, Document.hooks.ts: Added `<T, D>` generics ke `DocumentComponent` & render helpers; extracted `_renderDocumentHeader` & `_handleClosePreview` di container; improved JSDoc
8. Underwriting Data — Include Column —
UnderwritingDataDetail.config.tsx: Added "Include" column ke coverage detail table headers with `w-[13%]` width

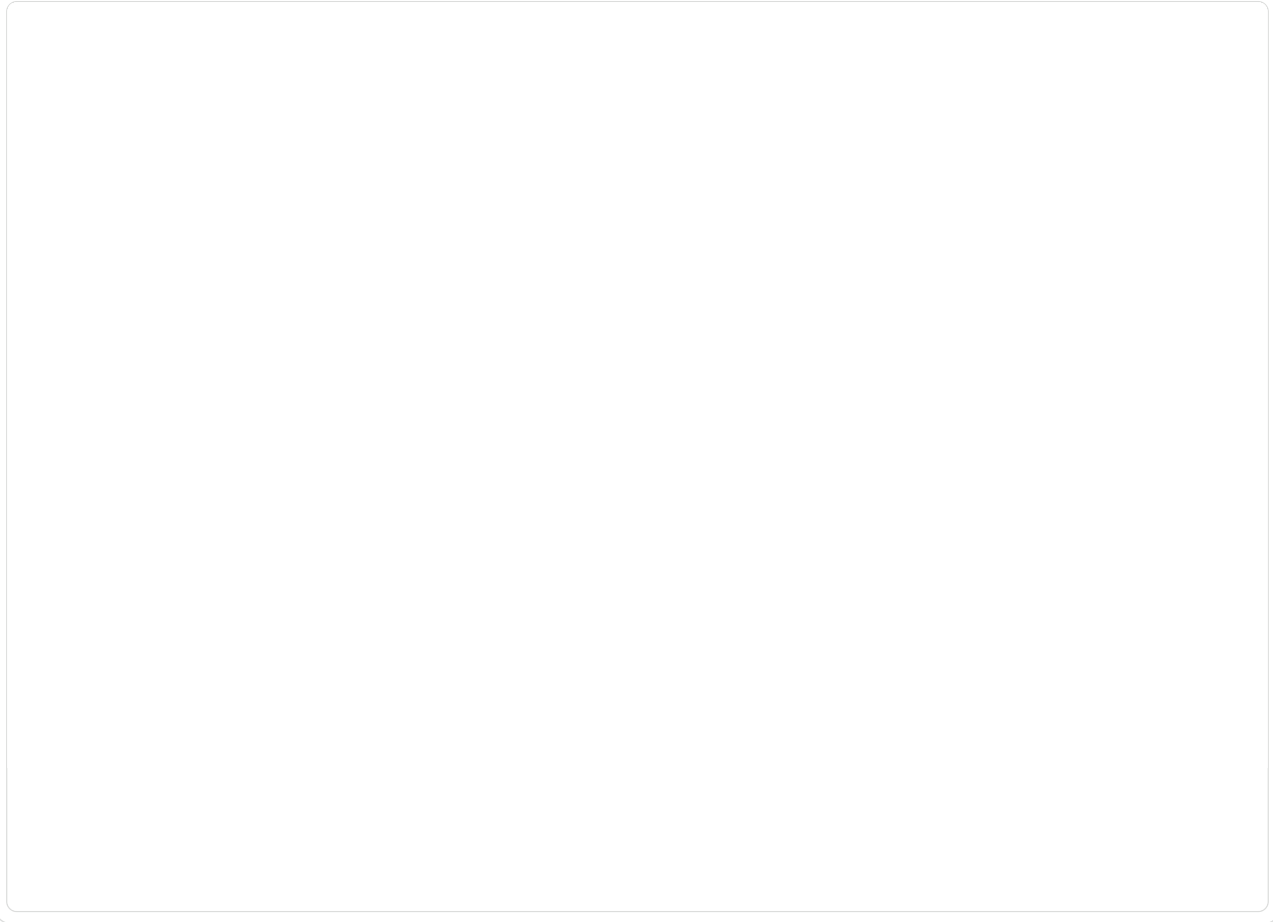


Screenshots:

success build, test, and linter

Git Branching Strategy (Git Flow)

- `feature/*` : For the development of specific features. Branch off from develop
- `development` : Main branch for new feature integration.
- `uat` : as the last static environment before the code enters the production stage
- `release/*` : Preparation for release to production.
- `release` : For release to production.



10. Enforcement

- **ES-Lint errors** must be resolved
- **Unit tests** must pass (minimum 70% coverage)
- **Typecheck** must pass
- The **Branch Name** must follow convention
- The **PR description** should be detailed

❌ Non-compliant code **WILL NOT** be **MERGED**

11. Conclusion

This convention ensures:

- Enterprise-level code quality
- Clear separation of concerns
- Scalable Next.js architecture
- Long-term maintainability